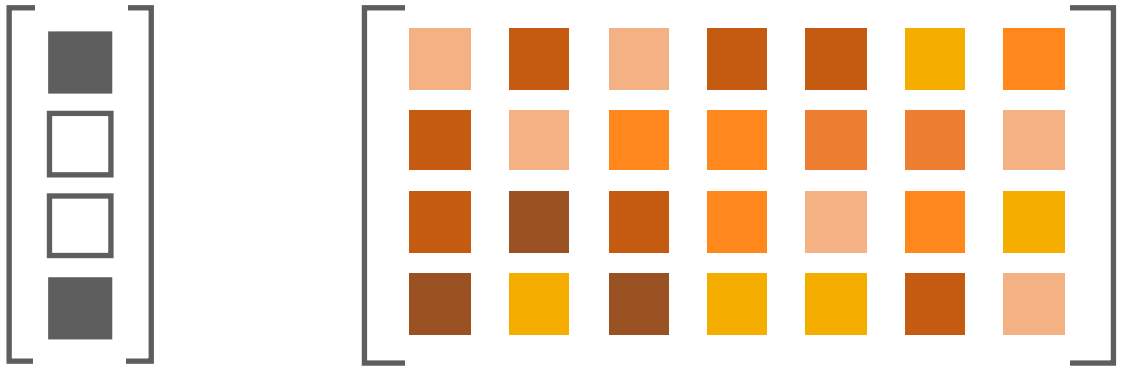
Introduction to Statistical Learning with Applications

CM5: Linear classifiers – Part 1

Pedro L. C. Rodrigues

Motivation

We are once again interested in supervised methods, but with an important novelty



$\mathbf{y} \in \{\square, \blacksquare\}^N$ $\qquad$ $\mathbf{X} \in \mathbb{R}^{N \times p}$

Problems with observations defined in a set are called classification problems

GOAL Decide the value of y_{test} given $\mathbf{x}_{\text{test}}$

In ISLA, we will focus on the fundamentals of classification methods

- Important tools from Probability
- The Bayes classifier
- Discriminative approaches
- Some practical issues

Important tools from probability theory

We will be needing several concepts from your classes in probability and statistics

- For any set $\mathcal{A} \subset \mathbb{R}^p$ and a pdf $p(\mathbf{x})$ we have that

$$\mathbb{E}_{\mathbf{X}}\Big[\mathbf{1}_{\mathcal{A}}(\mathbf{X})\Big] = \int_{\mathbb{R}^p} \mathbf{1}_{\mathcal{A}}(\mathbf{x})\; p(\mathbf{x})\mathrm{d}\mathbf{x} = \int_{\mathcal{A}} p(\mathbf{x})\mathrm{d}\mathbf{x} = \mathrm{Prob}\big(\mathbf{X} \in \mathcal{A}\big)$$

Important tools from probability theory

We will be needing several concepts from your classes in probability and statistics

- For any set $\mathcal{A} \subset \mathbb{R}^p$ and a pdf $p(\mathbf{x})$ we have that

$$\mathbb{E}_{\mathbf{X}}\Big[\mathbf{1}_{\mathcal{A}}(\mathbf{X})\Big] = \int_{\mathbb{R}^p} \mathbf{1}_{\mathcal{A}}(\mathbf{x})\; p(\mathbf{x})\mathrm{d}\mathbf{x} = \int_{\mathcal{A}} p(\mathbf{x})\mathrm{d}\mathbf{x} = \mathrm{Prob}\big(\mathbf{X} \in \mathcal{A}\big)$$

- For any function $g : \mathbb{R}^p \times \{0,1\} \to \mathbb{R}$ and joint pdf $p(\mathbf{x}, y)$ we have that

$$\mathbb{E}_{\mathbf{X},Y}[g(\mathbf{X},Y)] = \iint g(\mathbf{x},y)\; p(\mathbf{x},y)\mathrm{d}\mathbf{x}\mathrm{d}y = \iint g(\mathbf{x},y)\; p(y \mid \mathbf{x})p(\mathbf{x})\mathrm{d}\mathbf{x}\mathrm{d}y$$

$$= \int \left(\int g(\mathbf{x},y)\; p(y \mid \mathbf{x})\mathrm{d}y \right) p(\mathbf{x})\mathrm{d}\mathbf{x} = \mathbb{E}_{\mathbf{X}}\left[\mathbb{E}_{Y|\mathbf{X}}\Big[g(\mathbf{X},Y)\Big]\right]$$

Important tools from probability theory

- The conditional probability we will be using most often is written as

$$\mathrm{Prob}(Y = 1 \mid \mathbf{X} \in \mathcal{A}) = \frac{\mathrm{Prob}(Y = 1) \int_{\mathcal{A}} p(\mathbf{x} \mid Y = 1) \mathrm{d}\mathbf{x}}{\int_{\mathcal{A}} p(\mathbf{x}) \mathrm{d}\mathbf{x}}$$

with

$$\int_{\mathcal{A}} p(\mathbf{x}) \mathrm{d}\mathbf{x} = \mathrm{Prob}(Y = 1) \int_{\mathcal{A}} p(\mathbf{x} \mid Y = 1) \mathrm{d}\mathbf{x} + \mathrm{Prob}(Y = 0) \int \overbrace{p(\mathbf{x} \mid Y = 0)}^{\text{data likelihood}} \mathrm{d}\mathbf{x}$$

↳ Note that we are **very** careful when mixing continuous and discrete objects!

Important tools from probability theory

- The conditional probability we will be using most often is written as

$$\mathrm{Prob}(Y = 1 \mid \mathbf{X} \in \mathcal{A}) = \frac{\mathrm{Prob}(Y = 1) \int_{\mathcal{A}} p(\mathbf{x} \mid Y = 1) \mathrm{d}\mathbf{x}}{\int_{\mathcal{A}} p(\mathbf{x}) \mathrm{d}\mathbf{x}}$$

with

$$\int_{\mathcal{A}} p(\mathbf{x}) \mathrm{d}\mathbf{x} = \mathrm{Prob}(Y = 1) \int_{\mathcal{A}} p(\mathbf{x} \mid Y = 1) \mathrm{d}\mathbf{x} + \mathrm{Prob}(Y = 0) \int \overbrace{p(\mathbf{x} \mid Y = 0)}^{\text{data likelihood}} \mathrm{d}\mathbf{x}$$

Note that we are **very** careful when mixing continuous and discrete objects!

- The probability in a small ball of radius ε centered around $\mathbf{x}_0$ is

$$\mathrm{Prob}\Big(\mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big) \approx p(\mathbf{x}_0) \times \mathrm{Vol}\big(\mathcal{B}_\varepsilon(\mathbf{x}_0)\big)$$

Important tools from probability theory

The rigorous (and correct) way of expressing something like

The probability of Y being equal to 1 given that X is x_0

is to write

$$\mathrm{Prob}\Big(Y = 1 \mid \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big) = \mathrm{Prob}(Y = 1) \times \frac{\displaystyle\int_{\mathcal{B}_\varepsilon(\mathbf{x}_0)} p(\mathbf{x} \mid Y = 1)\mathrm{d}\mathbf{x}}{\displaystyle\int_{\mathcal{B}_\varepsilon(\mathbf{x}_0)} p(\mathbf{x})\mathrm{d}\mathbf{x}}$$

$$\approx \mathrm{Prob}(Y = 1) \times \frac{p(\mathbf{x}_0 \mid Y = 1) \times \mathrm{Vol}\big(\mathcal{B}_\varepsilon(\mathbf{x}_0)\big)}{p(\mathbf{x}_0) \times \mathrm{Vol}\big(\mathcal{B}_\varepsilon(\mathbf{x}_0)\big)}$$

Important tools from probability theory

The rigorous (and correct) way of expressing something like

The probability of Y being equal to 1 given that X is x_0

is to write

$$\mathrm{Prob}\Big(Y = 1 \mid \underline{\mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)}\Big) \approx \mathrm{Prob}(Y = 1) \times \frac{p(\mathbf{x}_0 \mid Y = 1)}{p(\mathbf{x}_0)}$$

For continuous random variables we always consider their probability to be in a set

But most books, articles, etc, will simply write 😟

$$p(y = 1 \mid \mathbf{x} = \mathbf{x}_0) = p(y = 1) \times \frac{p(\mathbf{x}_0 \mid y = 1)}{p(\mathbf{x}_0)}$$

- Important tools from Probability
- The Bayes classifier
- Discriminative approaches
- Some practical issues

The Bayes classifier

We want to find a function $f : \mathbb{R}^p \to \{0, 1\}$ such that it minimizes the probability error

$$\mathcal{L}(f) = \mathrm{Prob}\Big(Y \neq f(\mathbf{X})\Big) = \mathbb{E}_{\mathbf{X},Y}\Big[\mathbf{1}_{Y \neq f(\mathbf{X})}\Big] = \mathbb{E}_{\mathbf{X}}\Big[\mathbb{E}_{Y|\mathbf{X}}\big[\mathbf{1}_{Y \neq f(\mathbf{X})} \mid \mathbf{X}\big]\Big]$$

The Bayes classifier

We want to find a function $f : \mathbb{R}^p \to \{0, 1\}$ such that it minimizes the probability error

$$\mathcal{L}(f) = \text{Prob}\Big(Y \neq f(\mathbf{X})\Big) = \mathbb{E}_{\mathbf{X},Y}\Big[\mathbf{1}_{Y \neq f(\mathbf{X})}\Big] = \mathbb{E}_{\mathbf{X}}\Big[\mathbb{E}_{Y|\mathbf{X}}\big[\mathbf{1}_{Y \neq f(\mathbf{X})} \mid \mathbf{X}\big]\Big]$$

$$= \mathbb{E}_{\mathbf{X}}\Big[\text{Prob}(Y = 1 \mid \mathbf{X})\ \mathbf{1}_{f(\mathbf{X}) \neq 1} + \text{Prob}(Y = 0 \mid \mathbf{X})\ \mathbf{1}_{f(\mathbf{X}) \neq 0}\Big]$$

$$= \mathbb{E}_{\mathbf{X}}\Big[\text{Prob}(Y = 1 \mid \mathbf{X})\ \mathbf{1}_{f(\mathbf{X}) = 0} + \text{Prob}(Y = 0 \mid \mathbf{X})\ \mathbf{1}_{f(\mathbf{X}) = 1}\Big]$$

$$= \mathbb{E}_{\mathbf{X}}\Big[\underline{\Big(1 - \text{Prob}(Y = 0 \mid \mathbf{X})\Big)\ \mathbf{1}_{f(\mathbf{X}) = 0} + \Big(1 - \text{Prob}(Y = 1 \mid \mathbf{X})\Big)\ \mathbf{1}_{f(\mathbf{X}) = 1}}\Big]$$

we can focus on what is going on inside the expectation over $\mathbf{X}$

The Bayes classifier

We want to find a function $f : \mathbb{R}^p \to \{0, 1\}$ such that it minimizes the probability error

$$\mathcal{L}(f) = \mathbb{E}_{\mathbf{X}}\left[\Big(1 - \text{Prob}(Y = 0 \mid \mathbf{X})\Big)\ \mathbf{1}_{f(\mathbf{X})=0} + \Big(1 - \text{Prob}(Y = 1 \mid \mathbf{X})\Big)\ \mathbf{1}_{f(\mathbf{X})=1}\right]$$

Note that for each $\mathbf{X}$ two things can happen:

- $f(\mathbf{X}) = 0$ which contributes with $1 - \text{Prob}(Y = 0 \mid \mathbf{X})$ to the loss function
- $f(\mathbf{X}) = 1$ which contributes with $1 - \text{Prob}(Y = 1 \mid \mathbf{X})$ to the loss function

The optimal discriminant function for a test observation $\mathbf{x}$ is therefore

$$\boxed{f(\mathbf{x}_0) = \underset{k}{\text{argmax}}\ \text{Prob}\Big(Y = k \mid \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big)}$$

which is also known as the Bayes classifier

The Bayes classifier

- Example on a simulated dataset

The data points are defined in $\mathbb{R}^2$

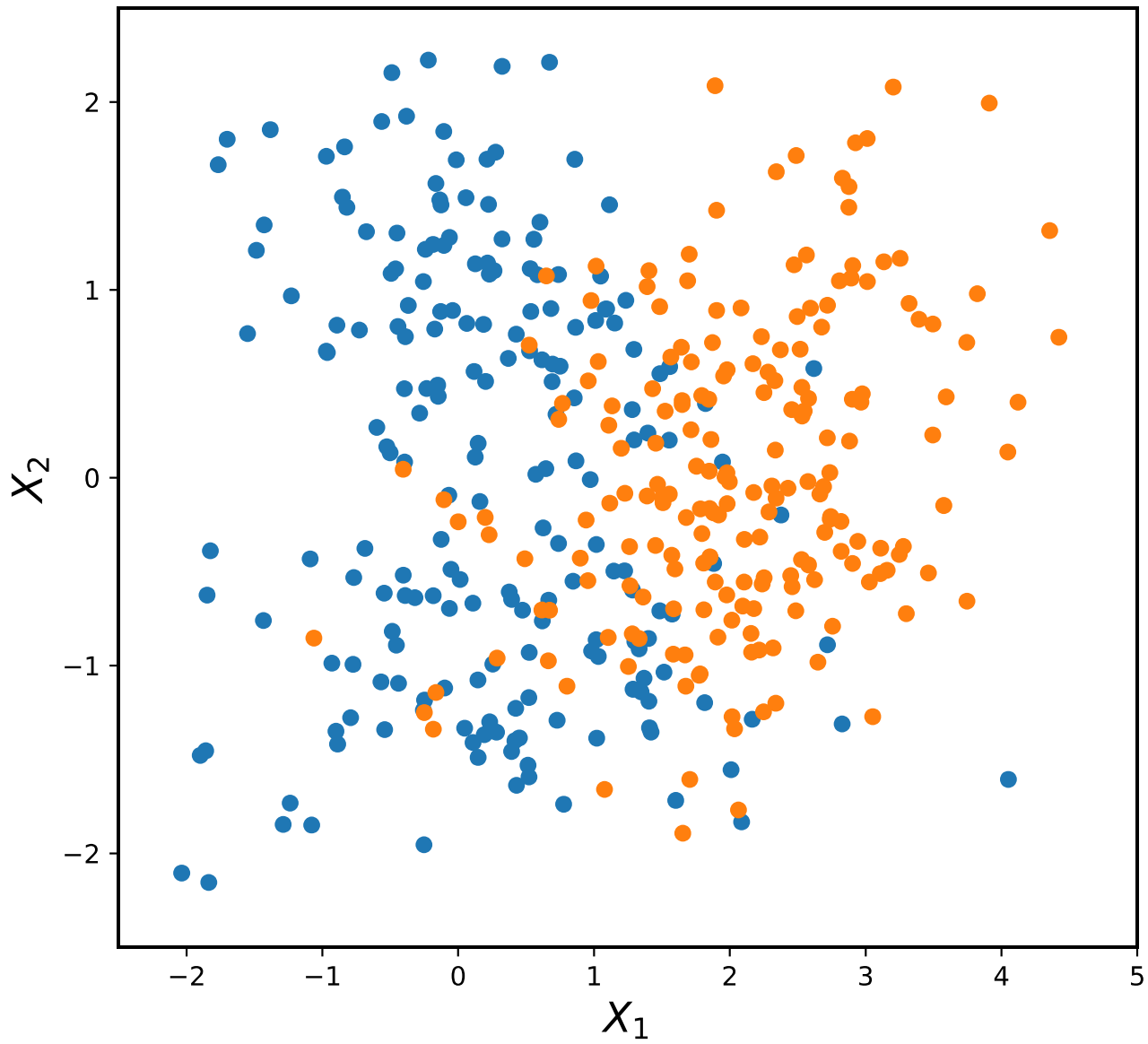
$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

and they are sampled as per

- $\mathbf{x} \mid Y = 0 \sim p_0(\mathbf{x})$
- $\mathbf{x} \mid Y = 1 \sim p_1(\mathbf{x})$

with

$$\text{Prob}(Y = 0) = \text{Prob}(Y = 1)$$



The Bayes classifier

- Example on a simulated dataset

The data points are defined in $\mathbb{R}^2$

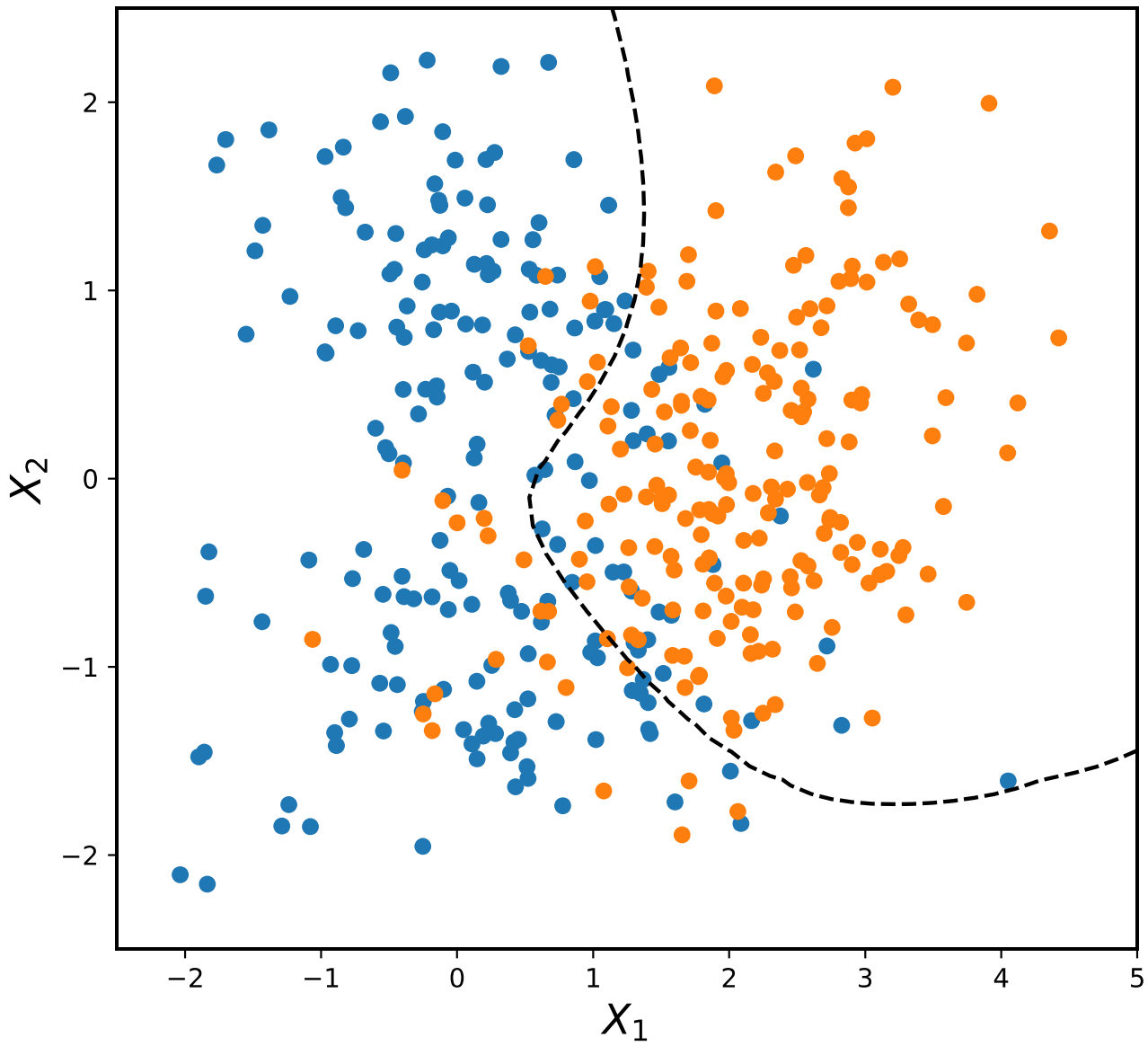
$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

and they are sampled as per

- $\mathbf{x} \mid Y = 0 \sim p_0(\mathbf{x})$
- $\mathbf{x} \mid Y = 1 \sim p_1(\mathbf{x})$

with

$$\text{Prob}(Y = 0) = \text{Prob}(Y = 1)$$



The Bayes classifier

- Example on a simulated dataset

The conditional probability for small ε is

$$\mathrm{Prob}\Big(Y = 1 \mid \underbrace{\mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)}_{\text{ball centered around } x_0}\Big) = \frac{1}{1 + \dfrac{p(\mathbf{x}_0 \mid Y = 0)}{p(\mathbf{x}_0 \mid Y = 1)}}$$

And the Bayes classifier is written as

$$f(\mathbf{x}) = \begin{cases} 1 & \text{if } p(\mathbf{x} \mid Y = 1) > p(\mathbf{x} \mid Y = 0) \\ 0 & \text{otherwise} \end{cases}$$

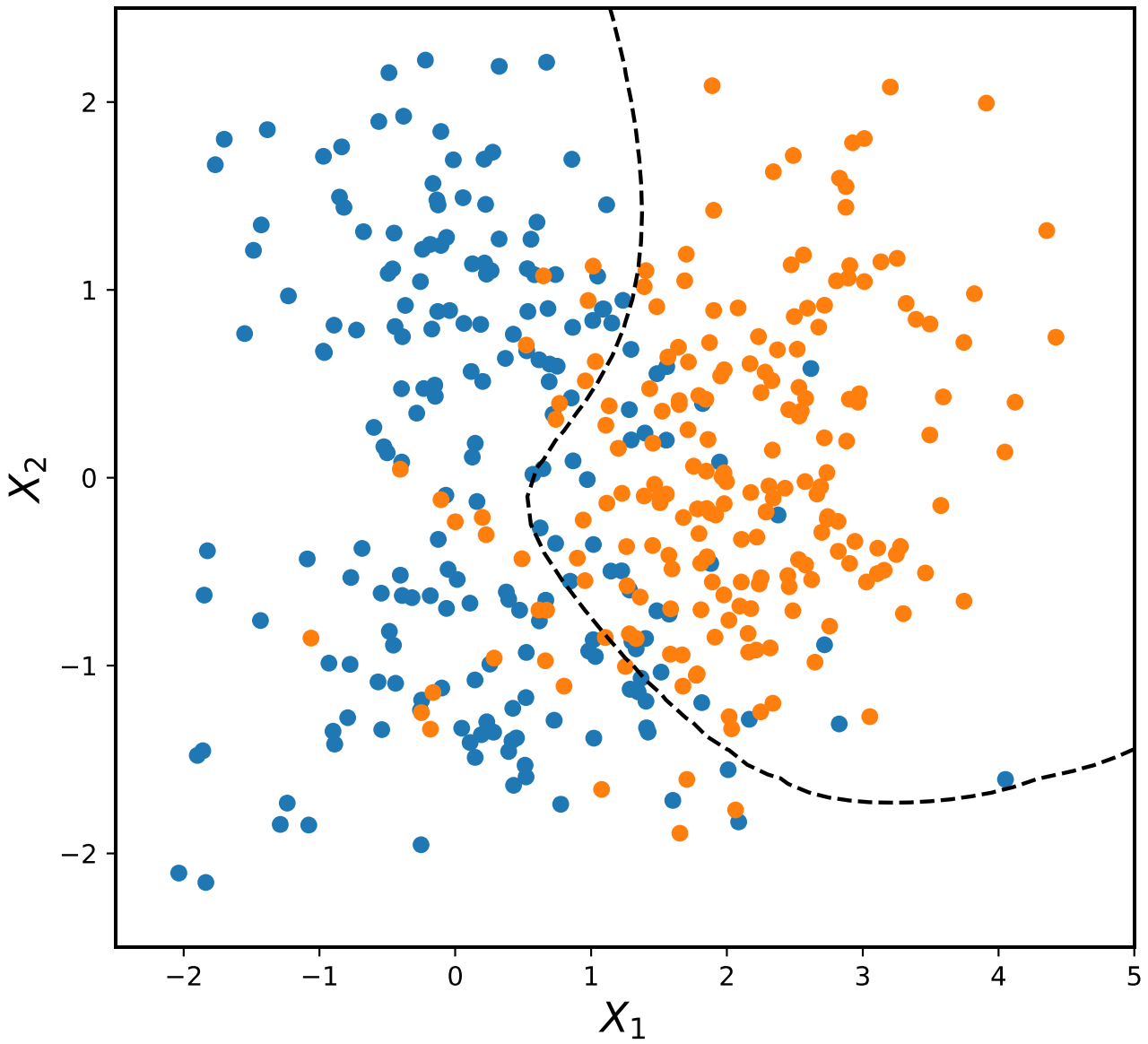
The boundary region is defined as $\mathcal{R} = \Big\{\mathbf{x} \in \mathbb{R}^p \mid p(\mathbf{x} \mid Y = 1) = p(\mathbf{x} \mid Y = 0)\Big\}$

The Bayes classifier

- Example on a simulated dataset

The error rate of the Bayes classifier is the minimal value that one may expect for any classifier applied to the dataset.

Is the problem solved then?



The Bayes classifier

- Example on a simulated dataset

The error rate of the Bayes classifier is the minimal value that one may expect for any classifier applied to the dataset.

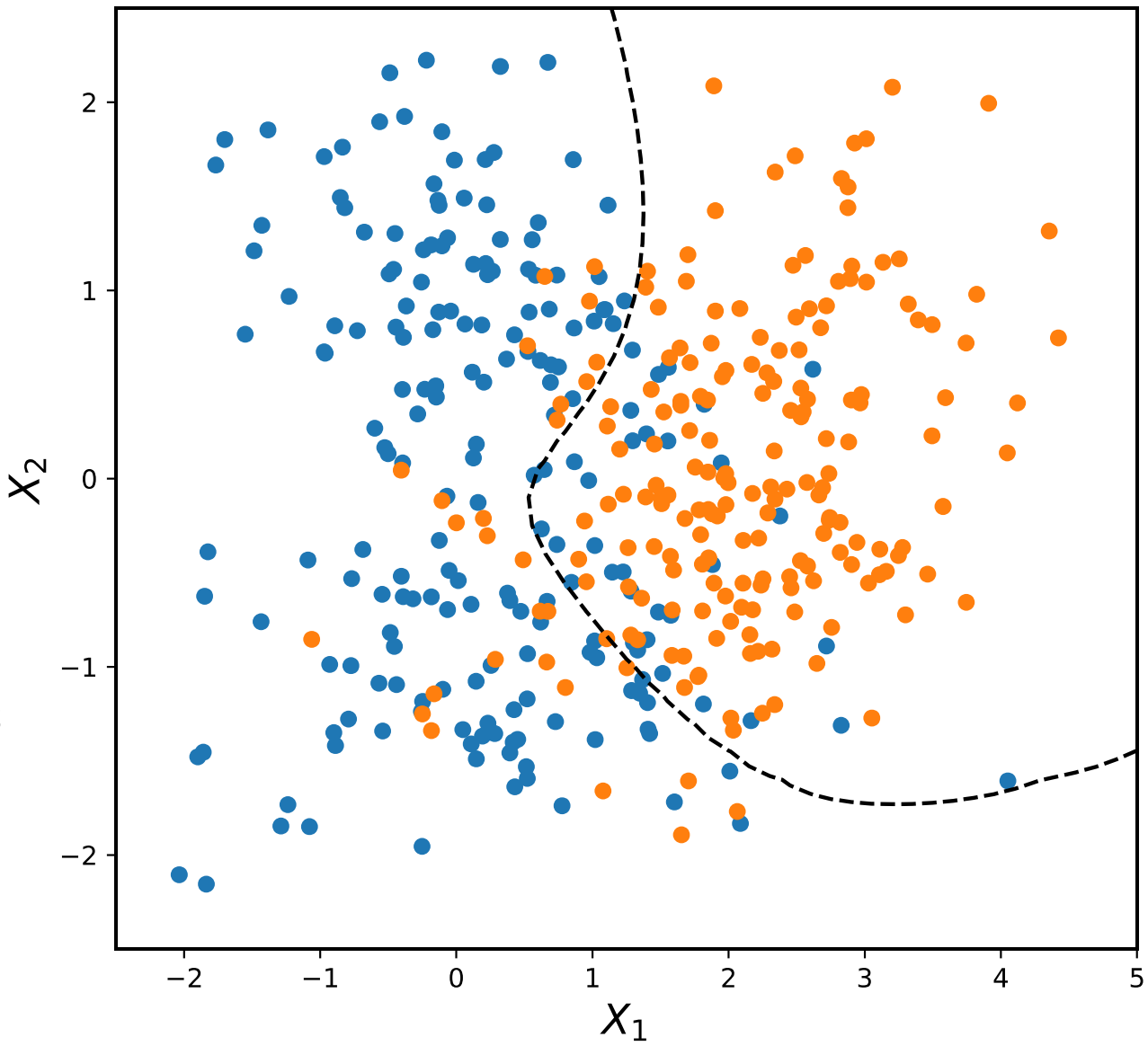
Is the problem solved then?

NO 😔

We never know the true data distributions

$$\mathrm{Prob}(Y \mid \mathbf{X}) \quad p(\mathbf{x}) \quad p(\mathbf{x}, Y)$$

So we end up approximating things...



- Important tools from Probability
- The Bayes classifier
- Discriminative approaches
- Some practical issues

Discriminative approaches

There are many strategies for approximating the Bayes classifier.

$$f(\mathbf{x}_0) = \underset{k}{\operatorname{argmax}} \operatorname{Prob}\Big(Y = k \mid \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big)$$

In this course we will focus on:

- K-nearest neighbours (kNN)
- Logistic regression
- Linear and quadratic discriminant analysis
- Naïve Bayes classifier
- Random forests
- Gradient boosted trees

Discriminative approaches

Generative approaches

Ensembling approaches

Discriminative approaches

- **K-nearest neighbours (kNN)**

Note first that for any test point $\mathbf{x}_0 \in \mathbb{R}^p$ we can always write

$$\mathrm{Prob}\Big(Y = 1 \mid \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big) = \mathbb{E}\Big[\mathbf{1}_{Y=1} \mid \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big]$$

$$= \int \mathbf{1}_{y=1}(y)\ \mathrm{Prob}\big(Y = y \mid \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\big)\mathrm{d}y$$

$$= \int \mathbf{1}_{y=1}(y)\ \left(\int_{\mathcal{B}_\varepsilon(\mathbf{x}_0)} p(\mathbf{x} \mid Y = y)\mathrm{d}\mathbf{x}\right) \mathrm{Prob}\big(Y = y\big)\mathrm{d}y \times \frac{1}{Z(\mathbf{x}_0)}$$

$$= \iint \mathbf{1}_{y=1}(y)\mathbf{1}_{\mathcal{B}_\varepsilon(\mathbf{x}_0)}(\mathbf{x})\ p(\mathbf{x}, y)\mathrm{d}\mathbf{x}\mathrm{d}y \times \frac{1}{Z(\mathbf{x}_0)}$$

$$\left[\ Z(\mathbf{x}_0) = \int_{\mathcal{B}(\mathbf{x}_0)} p(\mathbf{x})\mathrm{d}\mathbf{x}\ \right]$$

Discriminative approaches

- **K-nearest neighbours (kNN)**

Note first that for any test point $\mathbf{x}_0 \in \mathbb{R}^p$ we can always write

$$\mathrm{Prob}\Big(Y = 1 \,\Big|\, \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big) \approx \sum_{i=1}^{N} \mathbf{1}_{\mathcal{B}_\varepsilon(\mathbf{x}_0)}(\mathbf{x}_i)\mathbf{1}_{y=1}(y_i) \times \left(\sum_{i=1}^{N} \mathbf{1}_{\mathcal{B}_\varepsilon(\mathbf{x}_0)}(\mathbf{x}_i)\right)^{-1}$$

with $(\mathbf{x}_i, y_i) \sim p(\mathbf{x}, y)$

In kNN we approximate $\underline{\mathcal{B}_\varepsilon(\mathbf{x}_0) \approx \mathcal{N}_K(\mathbf{x}_0)}$ i.e. the set of K nearest neighbours of $\mathbf{x}$

This approximation is good when the data points are densely distributed

Discriminative approaches

- **K-nearest neighbours (kNN)**

In kNN we approximate $\mathcal{B}_\varepsilon(\mathbf{x}_0) \approx \mathcal{N}_K(\mathbf{x}_0)$ i.e. the set of K nearest neighbours of $\mathbf{x}$

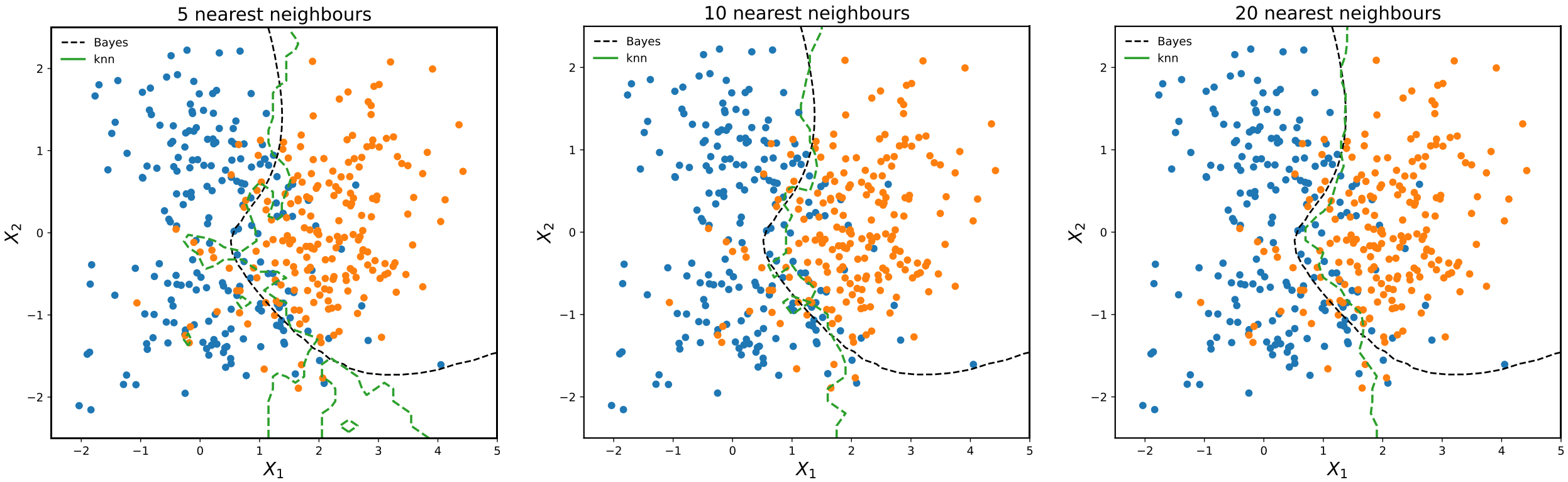
$$\boxed{\mathrm{Prob}\Big(Y = 1 \;\Big|\; \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big) \approx \frac{1}{K} \sum_{\mathbf{x}_i \in \mathcal{N}_K(\mathbf{x}_0)} \mathbf{1}_{y=1}(y_i)}$$

- kNN is a nonparametric classifier: it makes no assumptions about the data
  - ↳ It can be very flexible but not optimal when we know something about the data
- It works well in low dimensions, but suffers from the curse of dimensionality
  - ↳ You will see more of this with Razan
- The hyper-parameter K is usually chosen via cross-validation

Discriminative approaches

- **K-nearest neighbours (kNN)**

The classifier boundary gets smoother when K increases (Why?)

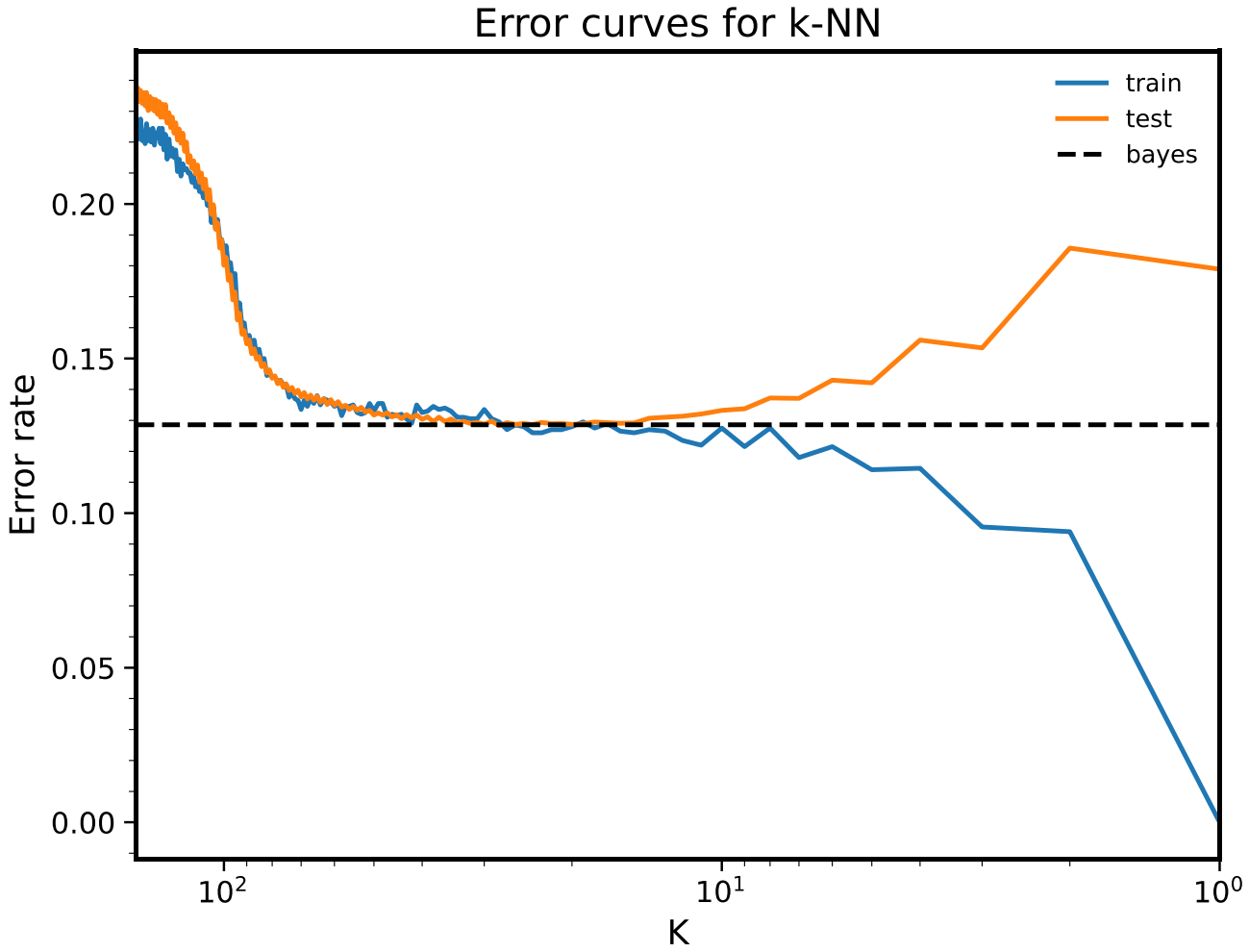


However, it does not look that much like the Bayes classifier boundary…

Discriminative approaches

- **K-nearest neighbours (kNN)**

The error of kNN usually evolves with the choice of K as below



Discriminative approaches

- **What about a parametric approach?**

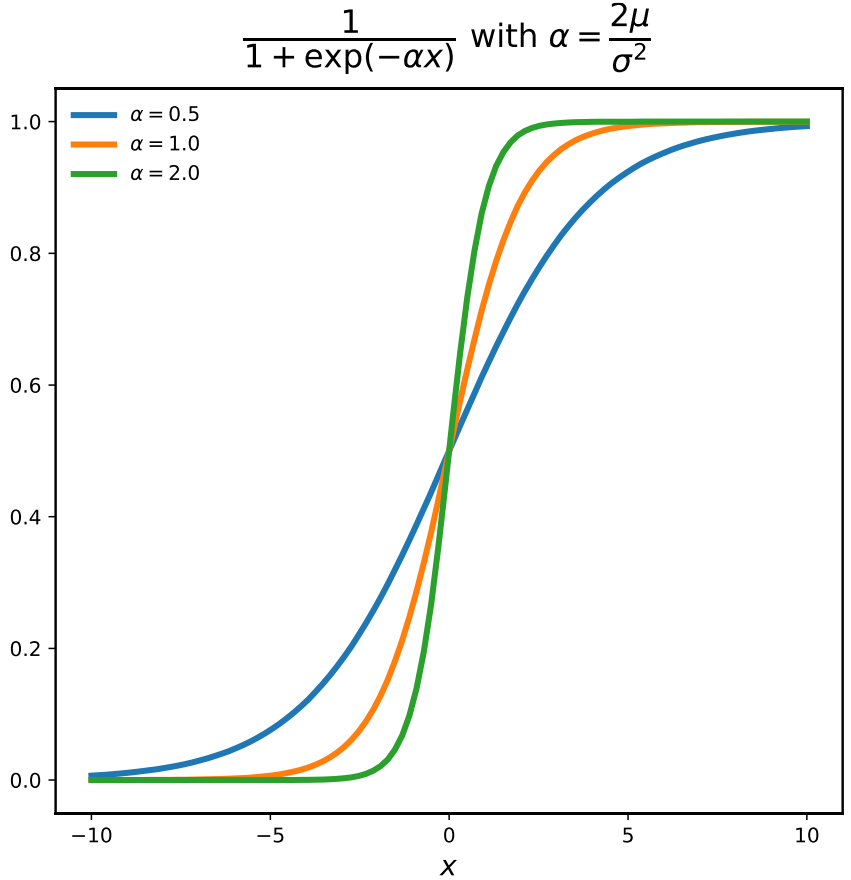
First, note that if we had $X \mid Y = 0 \sim \mathcal{N}(-\mu, \sigma^2)$ and $X \mid Y = 1 \sim \mathcal{N}(+\mu, \sigma^2)$

with $\mathrm{Prob}(Y = 0) = \mathrm{Prob}(Y = 1)$ then

$$\mathrm{Prob}\Big(Y = 1 \mid X \in \mathcal{B}_\varepsilon(x_0)\Big) = \frac{1}{1 + \exp\left(-\frac{2\mu}{\sigma^2} x_0\right)}$$

Therefore, the Bayes classifier is

$$f(x) = \mathbf{1}\left(\left[1 + \exp\left(-\frac{2\mu}{\sigma^2} x\right)\right]^{-1} \geq 0.5\right)$$



Discriminative approaches

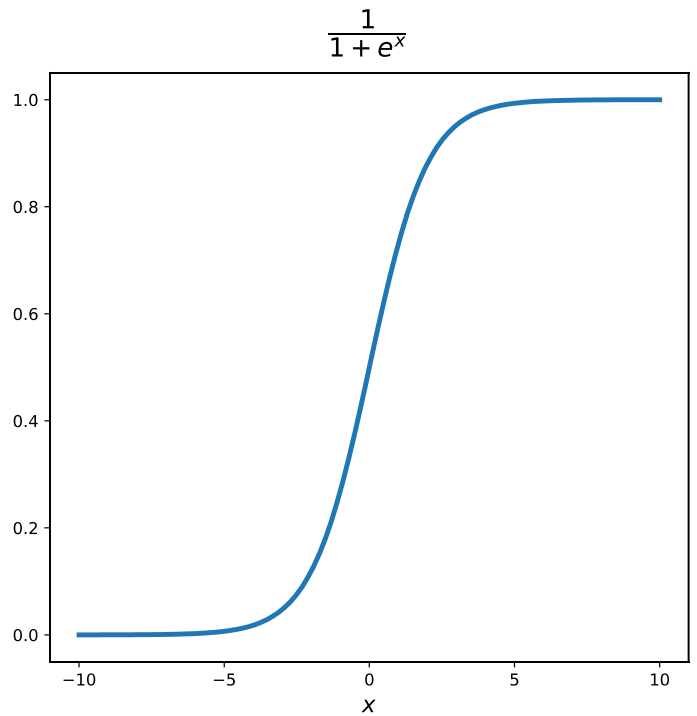
- **Logistic regression**

It sounds reasonable to propose a parametric approximator as

$$\mathrm{Prob}\Big(Y = 1 \mid \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big) \approx \frac{1}{1 + \exp\big(-\phi(\mathbf{x}_0)\big)}$$

where we will consider $\phi(\mathbf{x}) = \beta_0 + \sum_{k=1}^{p} \beta_k x_k$

and denote $\pi_\beta(\mathbf{x}) = \dfrac{1}{1 + \exp(-\phi(\mathbf{x}))}$



- Note that we have a quantity that is in [0, 1]

↳ What if we had used linear regression instead?

Discriminative approaches

- **Logistic regression**

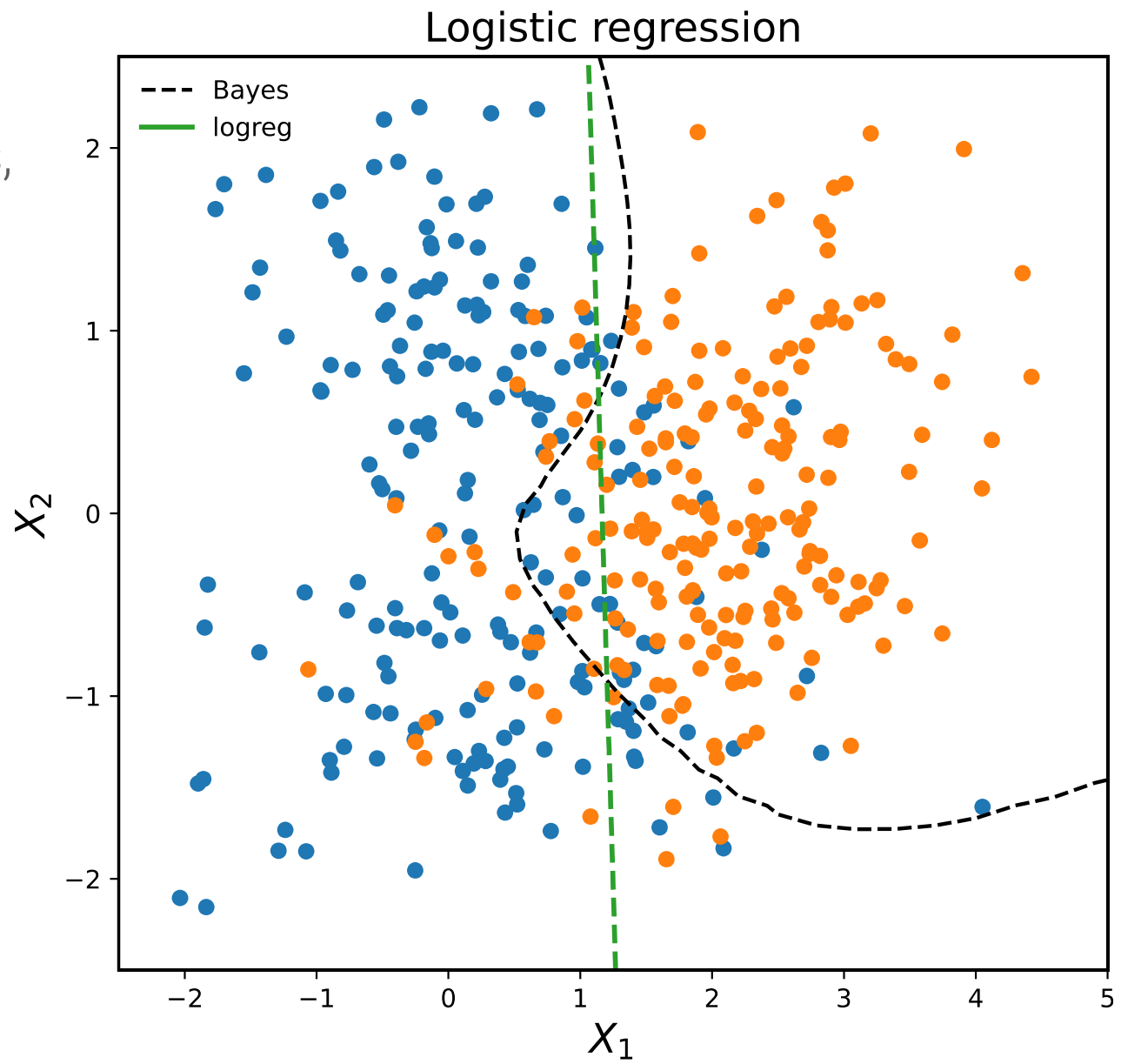
Despite being nonlinear in the parameters, the classification boundary of logistic regression is linear

↳ Which is why it is a linear classifier!

$$\frac{\exp(\phi(\mathbf{x}))}{1+\exp(\phi(\mathbf{x}))} = 0.5 \iff \phi(\mathbf{x}) = 0$$

$$\underline{\beta_0 + \sum_{k=1}^{p} \beta_k x_k = 0}$$

defines a separating hyperplane



Discriminative approaches

- **Logistic regression**

How do we estimate $\beta_0, \beta_1, \ldots, \beta_p$ in general?

Our goal is to have an approximator such that

$$\hat{P}(y \mid \mathbf{x}) \approx P(y \mid \mathbf{x}) = \mathrm{Prob}\Big(Y = y \mid \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x})\Big)$$

Note that since we're in the binary case, we can write

$$\hat{P}(1 \mid \mathbf{x}) = \pi_\beta(\mathbf{x}) \quad \text{and} \quad \hat{P}(0 \mid \mathbf{x}) = 1 - \pi_\beta(\mathbf{x})$$

We will look for parameters capable of minimizing a **statistical divergence**, i.e.

$$\beta^\star = \underset{\beta}{\operatorname{argmin}}\ \mathbb{E}_{\mathbf{X}}\Big[\mathrm{KL}\big(P(Y \mid \mathbf{X}) \| \hat{P}(Y \mid \mathbf{X})\big)\Big]$$

The approximator should be close to the target in average for every data point X

Discriminative approaches

- **Logistic regression**

How do we estimate $\beta_0, \beta_1, \ldots, \beta_p$ in general?

Expanding, we get

$$\mathbb{E}_{\mathbf{X}}\left[\mathrm{KL}\big(P(Y \mid \mathbf{X}) \| \hat{P}(Y \mid \mathbf{X})\big)\right] = \int \left(\sum_{y \in \{0,1\}} P(y \mid \mathbf{x}) \log \left(\frac{\hat{P}(y \mid \mathbf{x})}{P(y \mid \mathbf{x})} \right) \right) p(\mathbf{x}) \mathrm{d}\mathbf{x}$$

Which can be rewritten to give

$$\boxed{\beta^\star = \underset{\beta}{\operatorname{argmin}} \; -\frac{1}{N} \sum_{i=1}^{N} \Big(y_i \log\big(\pi_\beta(\mathbf{x}_i)\big) + (1 - y_i) \log\big(1 - \pi_\beta(\mathbf{x}_i)\big) \Big)}$$

Where the data points are $\{\mathbf{x}_i, y_i\}_{i=1}^{N}$ with $y_i \in \{0, 1\}$

This is also what people very often call the cross-entropy loss

Discriminative approaches

- **Logistic regression**

How do we estimate $\beta_0, \beta_1, \ldots, \beta_p$ in general?

Remember that

$$\pi_\beta(\mathbf{x}_i) = \frac{\exp(\beta_0 + \sum_{k=1}^p \beta_k x_{ik})}{1 + \exp(\beta_0 + \sum_{k=1}^p \beta_k x_{ik})}$$

Unfortunately, we no longer have a closed form solution for the parameters…

But we can proceed with a gradient descent procedure

$$\boldsymbol{\beta}^{\text{new}} = \boldsymbol{\beta}^{\text{old}} - \eta \nabla \mathcal{L}(\boldsymbol{\beta}) \quad \text{where} \quad \nabla \mathcal{L}(\beta) = \sum_{i=1}^N \Big(y_i - \pi_\beta(\mathbf{x}_i) \Big) \begin{bmatrix} 1 \\ \mathbf{x}_i \end{bmatrix}$$

which will converge to a global minimum since the loss function is concave

- Important tools from Probability
- The Bayes classifier
- Discriminative approaches
- Some practical issues

Some practical issues

What if we have more than just two classes?

The Bayes classifier is defined as before, but we compare the posteriors for all classes

$$f(\mathbf{x}_0) = \underset{k}{\operatorname{argmax}} \operatorname{Prob}\Big(Y = k \;\Big|\; \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big)$$

For logistic regression with C classes we use a softmax operator as below

$$\operatorname{Prob}\Big(Y = c \;\Big|\; \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big) = \frac{\exp(\beta_0^{(c)} + \sum_{k=1}^{p} \beta_k^{(c)} x_{0k})}{\sum_{\ell=1}^{C} \exp(\beta_0^{(\ell)} + \sum_{k=1}^{p} \beta_k^{(\ell)} x_{0k})}$$

so we have a set of p parameters for each class. This is multinomial regression

Some practical issues

How to use logistic regression in python?

LogisticRegression

```
class sklearn.linear_model.LogisticRegression(penalty='l2', *, dual=False,
tol=0.0001, C=1.0, fit_intercept=True, intercept_scaling=1, class_weight=None,
random_state=None, solver='lbfgs', max_iter=100, multi_class='deprecated',
verbose=0, warm_start=False, n_jobs=None, l1_ratio=None)
```

[source]

Logistic Regression (aka logit, MaxEnt) classifier.

This class implements regularized logistic regression using the 'liblinear' library, 'newton-cg', 'sag', 'saga' and 'lbfgs' solvers. **Note that regularization is applied by default.** It can handle both dense and sparse input. Use C-ordered arrays or CSR matrices containing 64-bit floats for optimal performance; any other input format will be converted (and copied).

Exercise from final exam in 2024

Consider a binary classification problem for which the data is generated as per

- $y_i = 0 \quad \Rightarrow \quad \mathbf{x}_i \sim 0.5\,\mathcal{N}(\boldsymbol{\mu}_0^{(a)}, \boldsymbol{\Sigma}_0) + 0.5\,\mathcal{N}(\boldsymbol{\mu}_0^{(b)}, \boldsymbol{\Sigma}_0)$
- $y_i = 1 \quad \Rightarrow \quad \mathbf{x}_i \sim \mathcal{N}(\boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1)$

where

$$\boldsymbol{\mu}_0^{(a)} = \begin{bmatrix} 0 \\ -1 \end{bmatrix} \quad \boldsymbol{\mu}_0^{(b)} = \begin{bmatrix} 0 \\ +1 \end{bmatrix} \quad \boldsymbol{\mu}_1 = \begin{bmatrix} \varepsilon \\ 0 \end{bmatrix}$$

$$\boldsymbol{\Sigma}_0 = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \quad \boldsymbol{\Sigma}_1 = \begin{bmatrix} 0.4 & 0 \\ 0 & 0.4 \end{bmatrix}$$

and $y_i \sim \mathcal{B}(p)$

Denote a set of N data points as $\mathcal{D}(N \mid \varepsilon, p)$

Exercise from final exam in 2024

(a) **Plot the data points** in $\mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{test}}$ using different colors to indicate classes of each data point and different symbols to indicate whether a point is from the train or the test set

$$\mathcal{D}_{\text{train}} = \mathcal{D}(50 \mid 2, 0.5) \text{ and } \mathcal{D}_{\text{test}} = \mathcal{D}(1000 \mid 2, 0.5)$$

(b) What is the **mathematical expression** for the optimal Bayes classifier? And for its boundary region? Do you expect it to be linear?

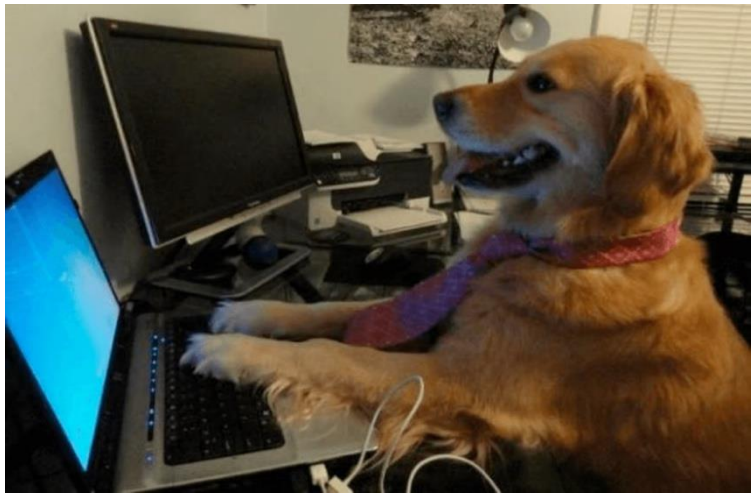
(c) Estimate the **error of the Bayes classifier** on the samples from $\mathcal{D}_{\text{test}}$. How do you expect this error to change as a function of epsilon?

Exercise from final exam in 2024

(a) **Plot the data points** in $\mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{test}}$ using different colors to indicate classes of each data point and different symbols to indicate whether a point is from the train or the test set

$$\mathcal{D}_{\text{train}} = \mathcal{D}(50 \mid 2, 0.5) \text{ and } \mathcal{D}_{\text{test}} = \mathcal{D}(1000 \mid 2, 0.5)$$

(b) What is the **mathematical expression** for the optimal Bayes classifier? And for its boundary region? Do you expect it to be linear?

(c) Estimate the **error of the Bayes classifier** on the samples from $\mathcal{D}_{\text{test}}$. How do you expect this error to change as a function of epsilon?

Let's do this! 🔥